

Week12_예습과제_장서연

* 모델 실습 파트는 ipynb파일

7장 트랜스포머

- Attention is All You Need (구글 브레인, 2017) 을 통해 소개된 신경망 아키텍처
- 병렬로 입력 시퀀스 처리
➡ 긴 시퀀스를 RNN(직렬)보다 훨씬 더 빠르고 효율적으로 처리
-  Self Attention : 입력 토큰 간의 관계 직접 처리, 이해
 - 기존 : 순차 처리(Sequential Processing) or 반복 연결(Recurrent Connections)
 - 모델이 재귀/합성곱 연산 없이 입력 토큰 간의 관계를 직접 모델링할 수 있음.
- 장기적인 종속성(기계 번역, 언어 모델링, 텍스트 요약) 포함 작업에 주로 사용, NLP에서 유명

트랜스포머 모델의 구조

- 주로 오토 인코딩 / 자기회귀 / 오토 인코딩+자기회귀의 조합

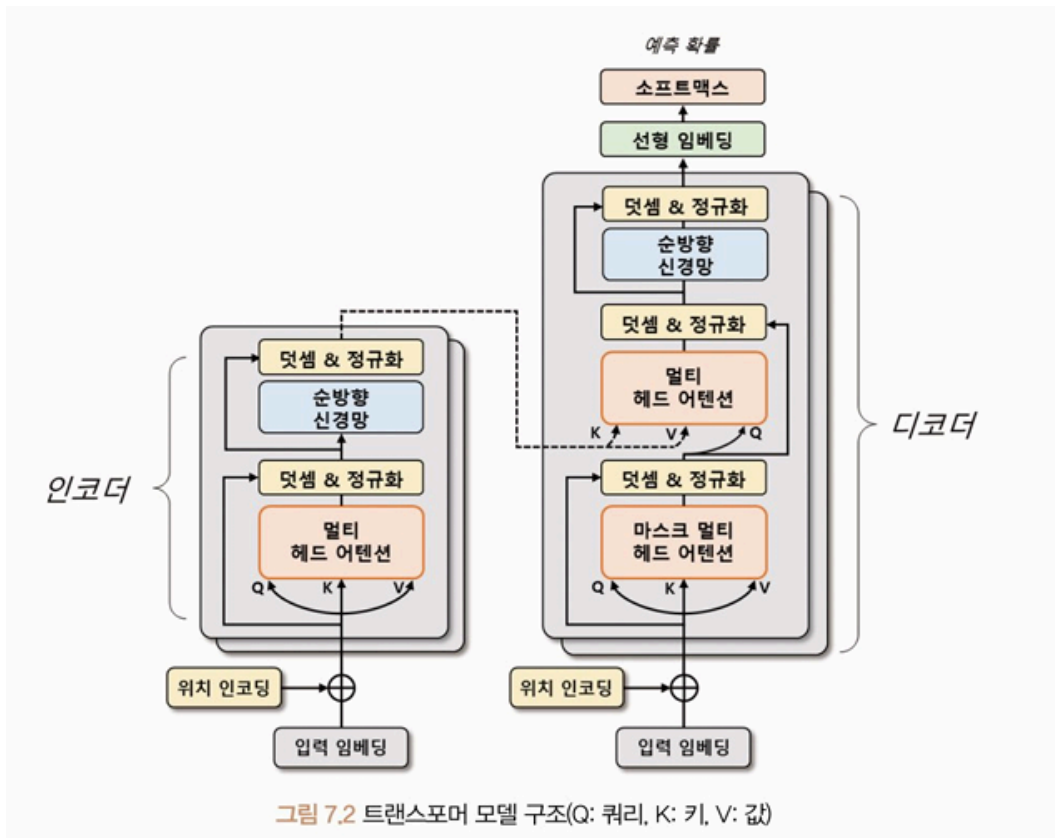
모델	학습구조	학습 방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더 + 디코더	오토 인코딩 + 자기회귀	양방향 + 단방향
ELECTRA	인코더 + 판별기	오토 인코딩 + 대체토큰 탐지	양방향
TS	인코더 + 디코더	오토 인코딩 + 자기회귀 + 다양한 NLP 작업	양방향

7.1 Transformer

- 어텐션 메커니즘만을 사용해 시퀀스 임베딩 표현
 - 인코더 : 입력 시퀀스를 임베딩해 고차원 벡터로 변환
 - 디코더 : 인코더의 출력을 입력으로 받아 출력 시퀀스 생성

- 인코더와 디코더 단어 사이의 상관관계를 계산해 중요한 정보에 집중
- 기존의 RNN 기반 모델보다 학습 속도가 빠르고, 병렬 처리가 가능해 대규모 데이터셋에서 높은 성능을 보임. 임베딩 과정에서 문장의 전체 정보를 고려해 문장의 길이가 길어지더라도 성능이 유지됨.

📌 트랜스포머 모델 구조



- 인코더와 디코더는 각각 N개의 트랜스포머 블록으로 구성됨.
- 트랜스포머 블록은 **멀티 헤드 어텐션**과 **순방향 신경망**으로 이루어짐.
- **멀티 헤드 어텐션**
 - 입력 시퀀스에서 쿼리, 키, 값 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법
 - 쿼리와 각 키의 유사도를 계산하고, 해당 유사도를 가중치로 사용해 값 벡터 합산함
 - 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체함. (입력 시퀀스의 단어사이 상호작용을 고려해 임베딩 벡터 갱신)
- **순방향 신경망**
 - 앞에서 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용.

- 여러개의 선형 계층으로 구성
- 입력벡터 * w(가중치) + b(편향) → 활성화 함수 → 학습 반복
- 입력 시퀀스 데이터를 소스와 타겟으로 나눠 처리
 - ex) 영어 → 한글 번역 / 영어(참조 언어) : 소스, 한글(생성 언어) : 타겟
- 인코더 : 소스 시퀀스 데이터를 위치 인코딩된 입력 임베딩으로 표현해 트랜스포머 블록의 출력 벡터 생성, 출력 벡터는 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성됨
- 디코더 : 마스크 멀티 헤드 어텐션을 사용해 타겟 시퀀스 데이터를 순차적으로 생성. 인코더의 출력 벡터 정보를 참조해 디코더 입력 시퀀스들의 관계를 고도화함
- 최종 출력 : 디코더 출력 벡터
 - 선형 임베딩으로 재표현
 - 이미지나 자연어 모델에 활용

7.1.1 입력 임베딩과 위치 인코딩

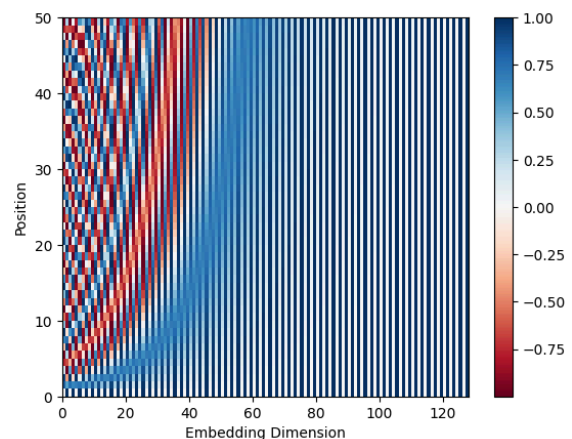
- 입력 임베딩 : 입력 시퀀스의 각 단어는 임베딩 처리돼 벡터형태로 변환됨.
- 위치 인코딩 : 단어의 순서 정보를 모델에 반영하기 위해 위치 정보를 임베딩 벡터에 추가하는 작업
(트랜스포머 : 병렬 구조 처리 - 단어의 순서 정보 제공 X)
 - 위치 인코딩 벡터는 sin, cos 함수 사용해 생성
 - 각 토큰의 위치를 각도로 표현해 sin함수와 cos함수로 위치 인코딩 벡터 계산
 - 결과 : 임베딩 벡터 + 위치 정보 → 최종 입력 벡터

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[: x.size(0)]
        return self.dropout(x)
```



✓ PositionalEncoding 클래스

- 입력 : 입력 임베딩 차원(d_{model}), 최대 시퀀스(max_len)
- 입력 시퀀스의 위치마다 $\sin$, $\cos$ 함수로 위치 인코딩 계산

수식 7.1 위치 인코딩

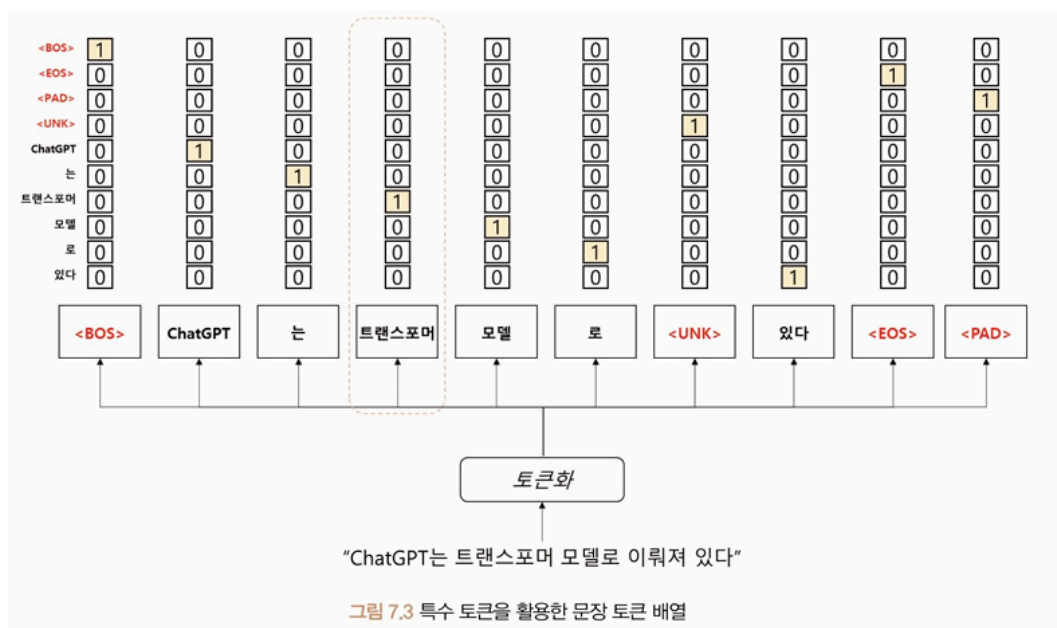
$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

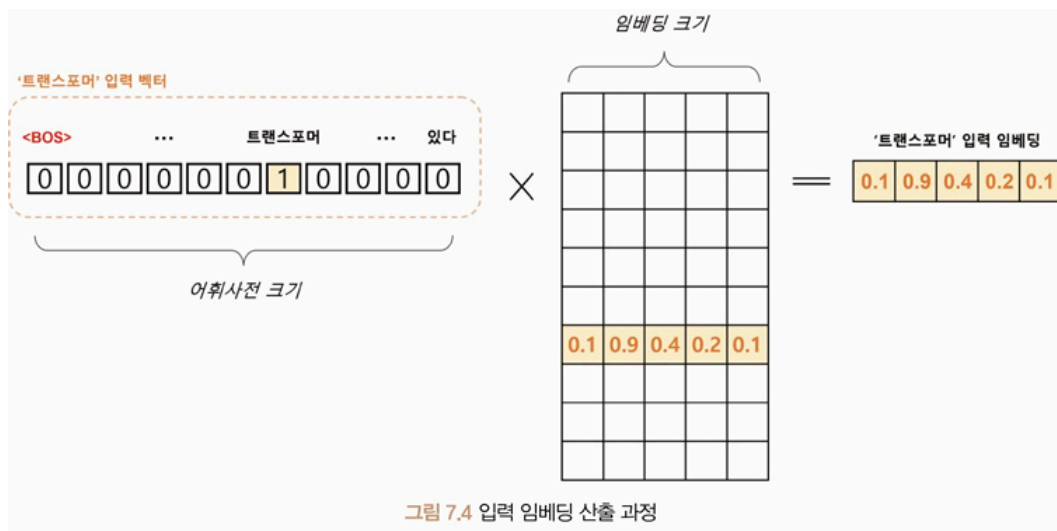
- pos : 입력 시퀀스에서 해당 단어의 위치
- i : 임베딩 벡터의 차원 인덱스 (짝수 : $\sin$, 홀수 : $\cos$)
- $1/10000^{2i/d_{model}}$: 각도 정보로 변환하기 위한 스케일링 인자, 각 위치에 대한 주기적 신호 생성
- pe : 텐서 차원 [최대 시퀀스, 1, 입력 임베딩의 차원]
- 출력 결과 : 위치별 임베딩 차원이 주기적인 값으로 구성됨 확인 가능
- 산출된 위치 인코딩 : 입력 임베딩과 더해진 후 드롭아웃 적용

7.1.2 특수 토큰

- 트랜스포머는 단어 토큰 이외의 특수 토큰을 활용해 문장을 표현함.
- 특수 토큰 : 입력 시퀀스의 시작과 끝 / 마스킹 영역으로 사용.

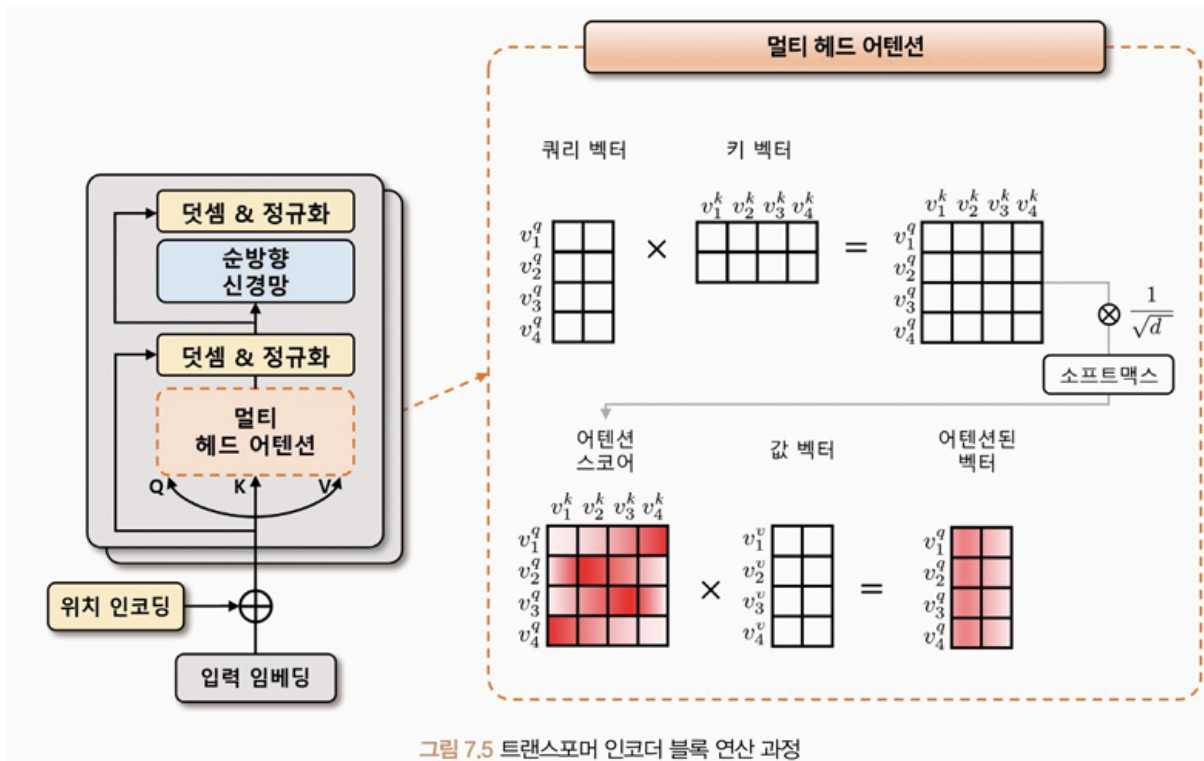


- BOS(문장의 시작), EOS(문장의 끝), UNK(모델이 이전에 본적없는 단어) 및 PAD(모든 문장을 일정한 길이로 맞추, 짧은 문장의 빈 공간을 채움) 토큰



- 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원핫 인코딩으로 표현
- 어휘 사전의 크기 = V , 입력 임베딩 차원 = d
- 원핫 벡터 = $[1, V]$, 임베딩 행렬 = $[V, d]$, 최종 변환 벡터 = $[1, d]$
- N 개의 문장이 최대 S 개의 토큰 길이를 가질 때
원핫 벡터 = $[N, S, V]$, 임베딩 행렬 = $[V, d]$, 최종 임베딩 텐서 = $[N, S, d]$
- 임베딩 텐서는 입력으로 사용되며, 트랜스포머의 모든 계층에서 공유되는 텐서로 사용됨.

7.1.3 트랜스포머 인코더



- 멀티 헤드 어텐션 + 순방향 신경망
- 입력 벡터+ 위치 임베딩 벡터 → 다음 계층(디코더 계층)으로 전달
- 입력 : 위치 인코딩이 적용된 소스 데이터의 입력 임베딩
 - 멀티 헤드 어텐션의 입력 : [N, S, d]
 - 입력 임베딩 선형 변환 후 : 3개의 임베딩 벡터 - 쿼리(Q), 키(K), 값(V)
 - 쿼리 벡터
 - 현재 시점에서 참조하고자 하는 정보의 위치를 나타냄
 - 인코더의 각 시점마다 생성
 - 현재 시점에서 질문이 되는 벡터, 이 벡터를 기준으로 다른 시점 정보 참조
 - 키 벡터
 - 쿼리벡터와 비교되는 대상
 - 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
 - 인코더의 각 시점에서 생성
 - 값 벡터
 - 어텐션 스코어(쿼리 벡터 + 키 벡터)의 가중치

수식 7.2 어텐션 스코어 계산식

$$\text{score}(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

- 어텐션 스코어 * 값 벡터 → 셀프 어텐션된 스코어 맵
- 출력 과정
 - 입력 : [N, S, d]
 - 멀티 헤드 : 셀프 어텐션 k번 수행 → k 개의 셀프 어텐션 벡터 생성
 - 헤드에 대한 차원 축소 생성 : [N, k, S, d/k] → 셀프 어텐션된 [N, S, d/k]
 - 셀프 어텐션 벡터 k개를 임베딩 차원 축으로 병합 : [N, S, d]로 출력
 - 덧셈 - 입력값 + 출력값
 - 기울기 소실 완화
 - 정규화 - 임베딩 차원 축으로 계층 정규화
- 순방향 신경망
 1. 선형 임베딩 + ReLU 인공 신경망
 2. 1차원 합성곱→ 덧셈 & 정규화
- 인코더의 출력 벡터가 디코더에서 사용,
디코더의 멀티 헤드 어텐션 모듈에서 키, 값 벡터가 됨.

7.1.4 트랜스포머 디코더

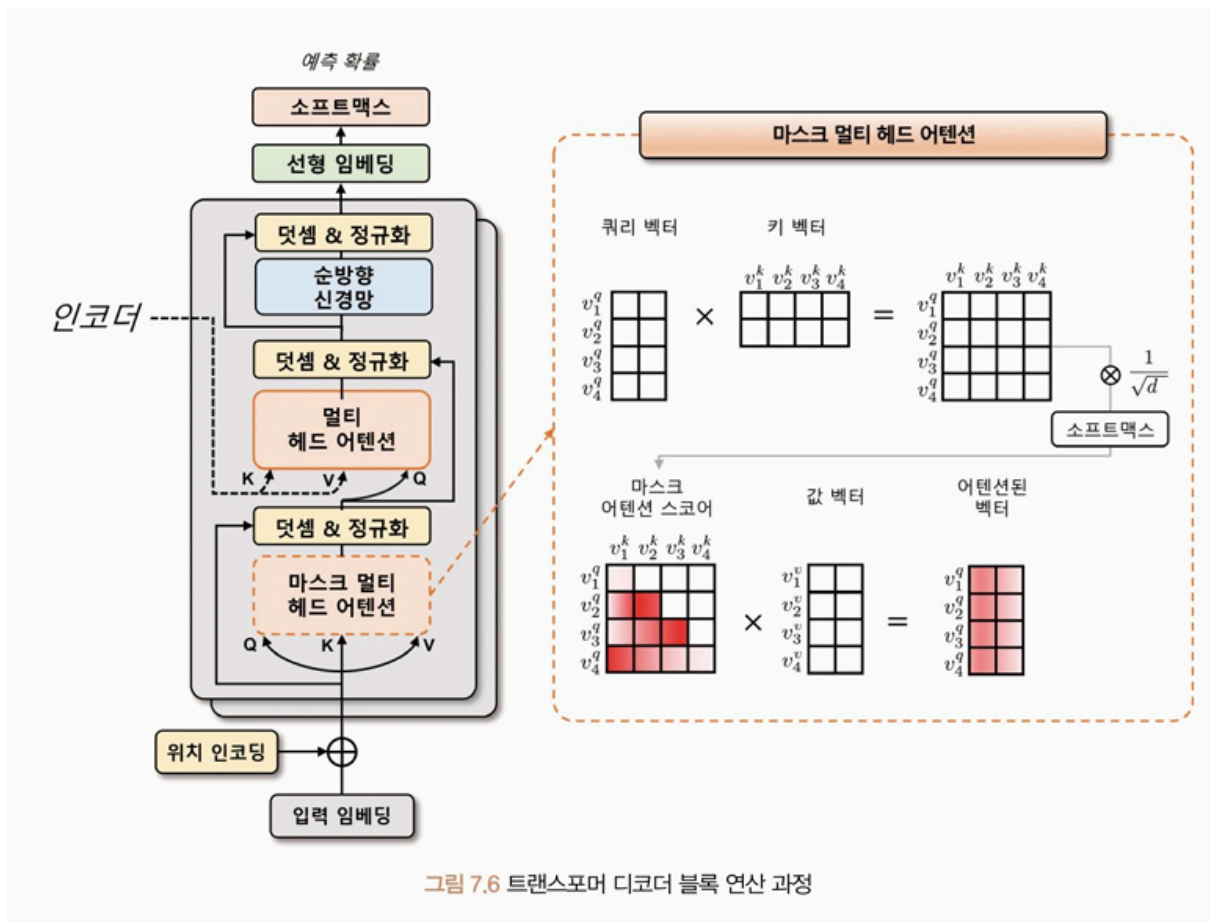


그림 7.6 트랜스포머 디코더 블록 연산 과정

- 입력 : 위치 인코딩이 적용된 타겟 데이터의 입력 임베딩
- 마스크 멀티 헤드 어텐션 모듈
 - 인코더의 멀티 헤드 어텐션 모듈에서 인과성을 반영
 - 어텐션 스코어 맵 계산 시 1st 쿼리벡터가 1st 키 벡터만을 볼 수 있게 마스크를 씌움
 - 2nd 쿼리 벡터는 1st, 2nd 키 벡터를 보게 마스크를 씌움
 - n번째 쿼리 벡터는 n-1번째 이전의 키벡터만을 볼 수 있음
 - 현재 위치 이전의 단어들만 참조할 수 있게 돼 인과성을 보장
 - 마스크 영역에 매우 작은 값인 -inf 마스크를 더함
 - 해당 영역의 어텐션 스코어 값이 -이 됨.
 - softmax 계산 시 해당 영역의 어텐션 가중치가 0이 됨.
 - 마스크 영역 이전의 입력 토큰들에 대한 정보를 참조하지 않게 됨.
 - 쿼리 벡터 : 타겟 데이터, 키&값 데이터 : 인코더의 소스 데이터
- 출력

- 쿼리 + 키 + 값 → 어텐션 스코어 → softmax → 어텐션 가중치 * 값 벡터 → 출력 벡터
- 다음 트랜스포머 디코더 블록의 입력으로 전달됨.
 - 이전 시점의 정보를 현재 시점에서 활용할 수 있게 됨.
 - 마지막 디코더 블록의 [N, S, d]에 선형 변환/softmax 적용 → 각 타깃 시퀀스 위치마다 예측 확률 계산 가능

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

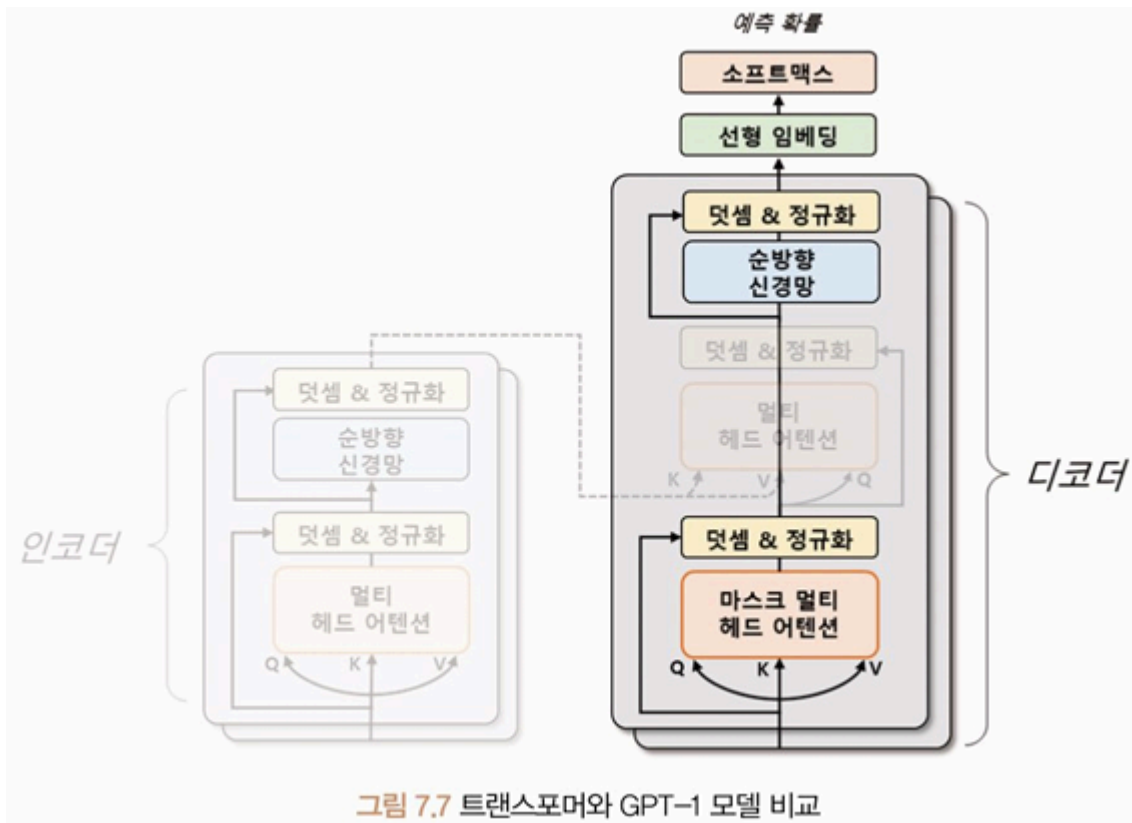
- 타깃 데이터를 추론할 때 토큰 또는 단어를 순차적으로 생성시키는 모델 (순차 : PAD 토큰/빈값)

7.2 GPT(Generative Pre-trained Transformer)

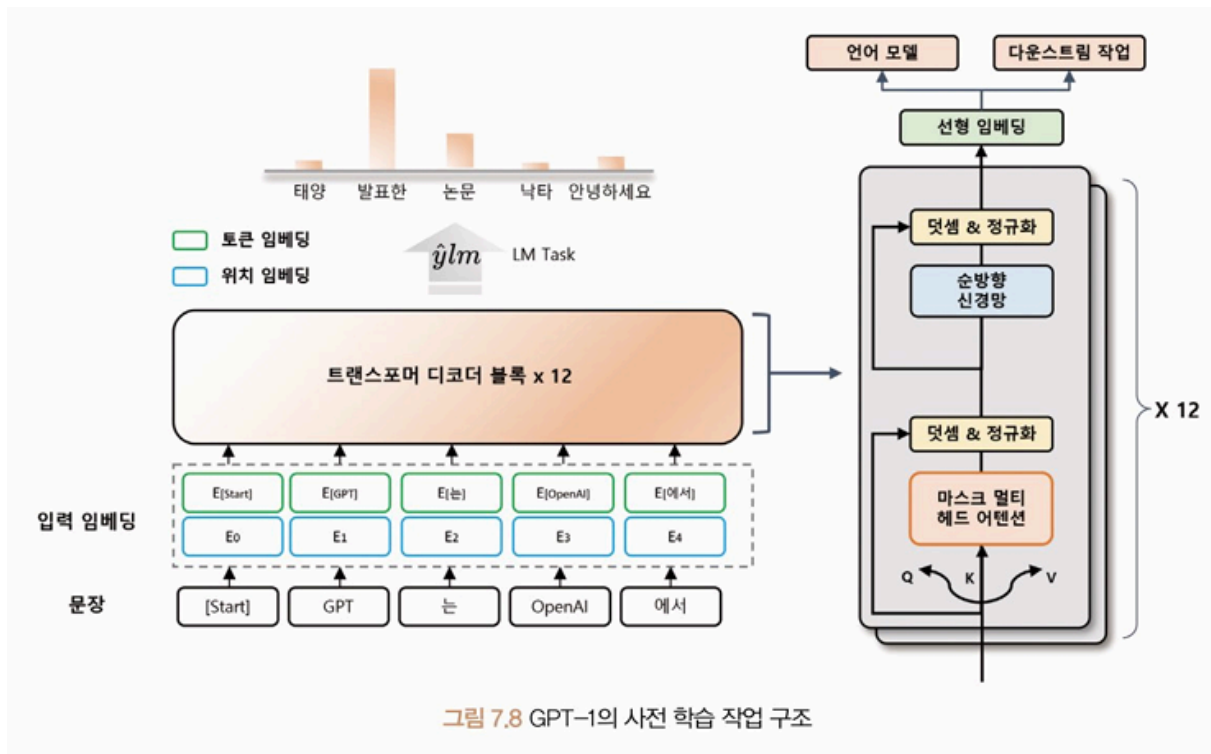
- ELMo(Embeddings from Language Model)과 같이 대규모 말뭉치로 사전학습된 모델
- 트랜스포머의 디코더를 여러층 쌓아 만든 언어 모델
 - 트랜스포머의 인코더 : 입력 문장의 특징 추출

- 트랜스포머의 디코더 : 입력 문장과 출력 문장 간의 관계 이해, 출력 문장 생성
→ 디코더를 쌓으면 자연어 생성 같은 모델링 작업에서 더 높은 성능 발휘
- 시리즈들은 거의 동일한 트랜스포머 구조 사용
- 버전이 갱신될수록 더 많은 트랜스포머 계층과 데이터를 활용해 학습됨.
 - GPT 1 → GPT 2 → GPT 3
매개변수 수 : 1억 2천 → 15억 → 1750억
 - GPT 4 : 이미지 데이터 처리에도 능해짐
 - GPT - Neo, GPT - J 등 다양한 시리즈의 모델들이 개발되는 중임
- RLHF(Reinforcement Learning from Human Feedback)
 - 매개변수를 줄이기 위해 도입
 - 인간의 피드백을 이용하는 강화학습 방법
 - 60억 개의 가중치로도 뛰어난 성능을 낼 수 있게 됨.

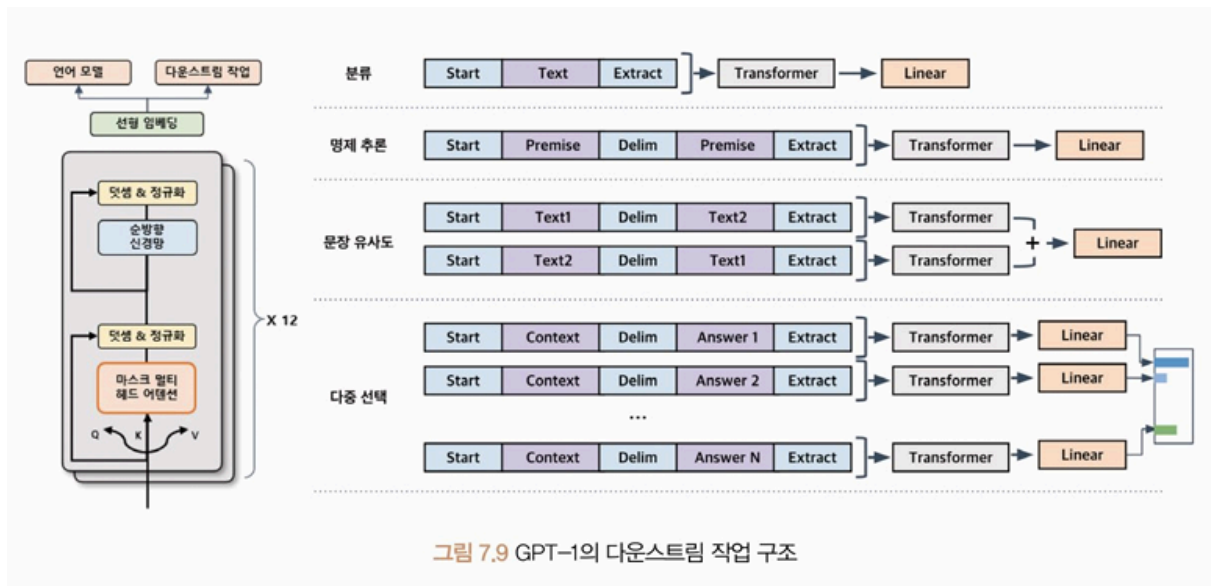
7.2.1 GPT-1



- 단방향(Unidirectional) 언어모델
 - 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측
 - 이전 단어들을 차례대로 입력하며 다음 단어 예측
 - 모델의 계산량 ↓, 모델의 크기 ↓
- 트랜스포머의 디코더 부분만 사용
 - 모델의 매개변수 ↓, 높은 성능.
- 인코더 - 디코더 간 어텐션 연산 제외



- 디코더의 2번째 다중헤드 어텐션 사용 ❌
- 12개의 헤드를 가진 트랜스포머의 디코더를 12층 쌓아 모델을 구성
- 1억 2천만 개의 매개 변수
- BookCorpus 데이터셋을 이용해 언어모델 사전 학습
 - 입력 문장을 임의의 위치까지 보여주고 뒷문장을 자동예측하게 함.
 - 비지도 학습(별도의 레이블 필요 ❌) : 컴퓨팅 자원만 필요
- 입력 임베딩 : 토큰 임베딩 + 위치 임베딩
 - 토큰 임베딩 : 각 토큰을 고정된 차원의 벡터로 매핑
 - 위치 임베딩 : 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터 매핑
- fine-tuning을 위한 다운스트림 작업에도 GPT1은 입출력 구조 변경 없이 언어 모델을 보조 학습해 사용 가능
 - 보조 학습 : 모델이 주어진 주요 학습 작업 외에도 다른 작은 부가적인 학습 작업을 동시 수행하는 것
 - 보조 작업에서 학습한 정보를 주요 작업에 전달해 성능 향상 가능



- 다운스트림 작업
 - 원하는 목표에 맞게 모델을 미세하게 fine-tuning
 - 추가 손실 함수 필요
 - 특수 토큰 사용
 - 시작 : <start>, 문장의 경계 : <delim>, 문장의 마지막 : <extract>

7.2.2 GPT-2

- 입력 문장 길이 : 512 → 1024
- 디코더 계층 수 : 12 → 48
- 매개변수 수 : 1억 200만 → 15억

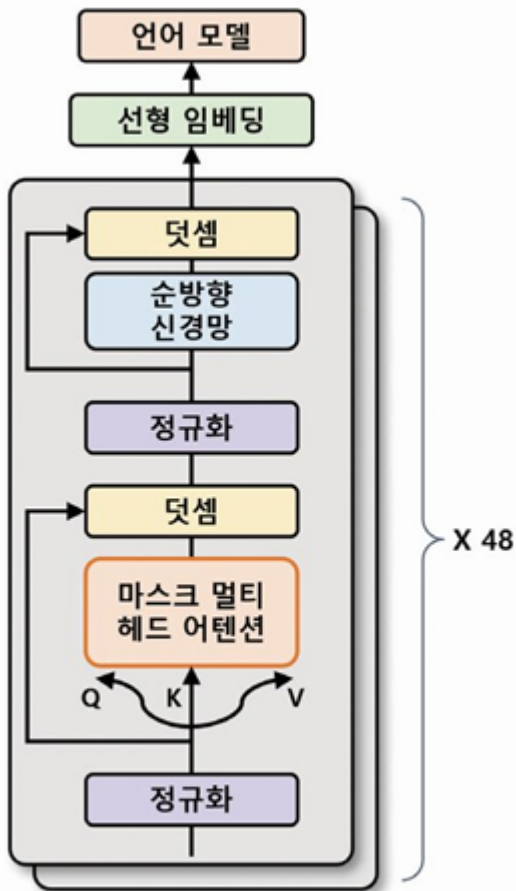
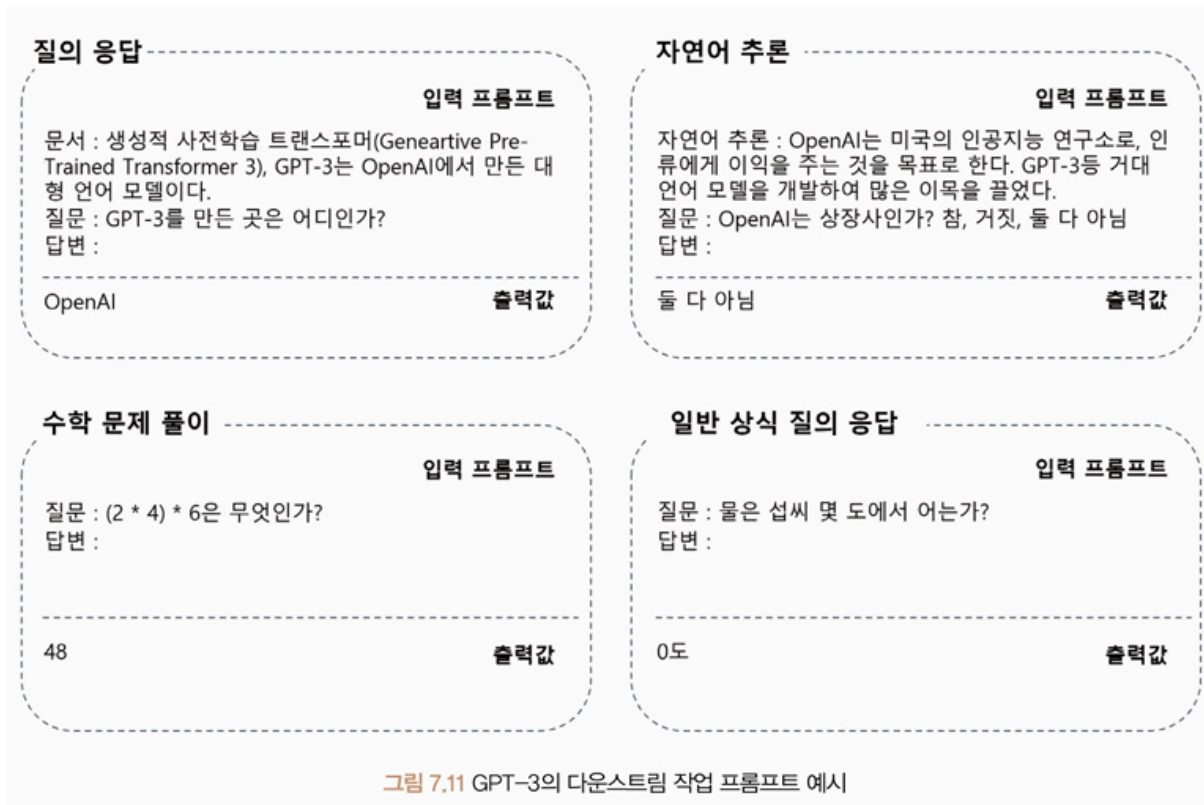


그림 7.10 GPT-2의 모델 구조

- 이전보다 더 많은 데이터 이용해 사전 학습
- 더욱 자연스러운 문장생성
- 제로-샷 학습 가능, 별도의 fine-tuning 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습 과정에서 특정한 형식의 데이터 입력

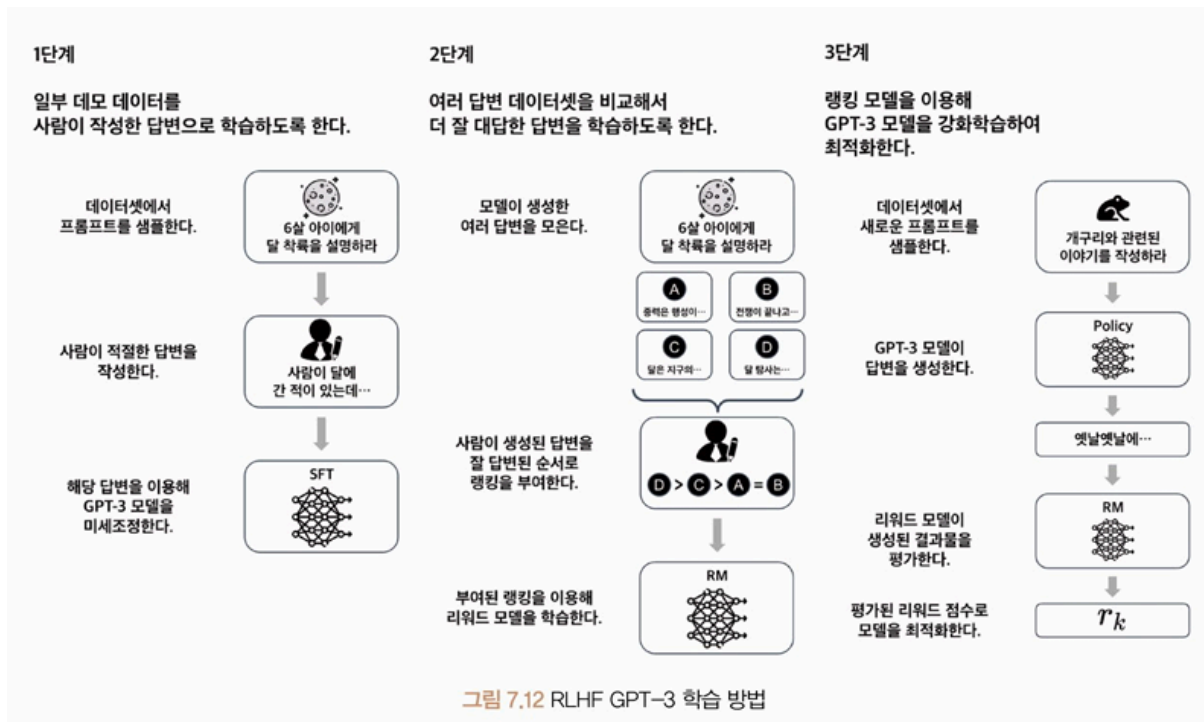
7.2.3 GPT-3

- 모델 구조는 이전과 거의 동일
- 매개변수 수 : 약 116배 증가
- 디코더 층 수 : 48 → 96, 96개의 헤드를 가진 멀티헤드 어텐션 사용
- 멀티헤드 어텐션 : 희소 어텐션과 일반 어텐션 섞어서 사용 (연산량 ↓)
- 토큰 임베딩 크기 : 1600 → 12888 (입력 토큰 수 : 1024 → 2048)
- 프롬프트 형식으로 작업 수행 (2랑 똑같음)
- 다운스트림 작업 학습 ❌, but 다운스트림 작업에서도 성능 ↑



- 높은 일반화 능력
- 질의응답, 수학문제 풀이, 자연어 추론 등과 같은 기존 자연어 처리 분야에서 이전의 기술적 한계 극복
- 규모 ⬆️ : 매우 느리고 비용 많이 듦.
- 사전 학습된 데이터에 기반해 작동 : 데이터가 편향 or 부정확 → 오류
- 상식적인 추론 능력 ⬇️
- 검토, 검증, 저작권 주의

7.2.4 GPT-3.5



- 새로운 학습 방법인 RLHF 도입
 1. 임의의 프롬프트 가져옴 → 사람이 적절한 답 작성
 2. 모델이 생성한 문장과 사람 답 함께 평가
 3. 자연스러움 : 보상, 부자연스러움 : 보상 ❌
 → 강화학습을 통해 모델 학습, 지도 미세 조정(Supervised Fine-Tuning, SFT)
- 학습데이터 양 고려시 사람이 모든 답변 불가
 - 보상 모델 학습
 - 보상 모델로 사람이 평가, 랭킹 부여
 - 랭킹을 보상 모델의 입력으로 사용

7.2.5 GPT-4

- Hallucination 현상
- 멀티모달 모델
- 입력 토큰 수 : 4096 → 32768
- 단어 수 : 3000 → 25000